

DiffraScout: An auditable workflow from candidate phases to indexed powder diffraction and elasticity references

Delun Gong

12 August 2026

Summary

Powder X-ray diffraction studies often begin with a reference-building task: researchers assemble plausible crystal structures, calculate where their diffraction peaks should occur, and retain enough context to judge whether those references are suitable for a given experiment. **DiffraScout** is an open-source Python package that makes this task reproducible. It accepts an alloy grade, chemical system, Materials Project identifier, or local Crystallographic Information File (CIF); discovers and ranks candidate phases when requested; validates the acquired structures; calculates indexed theoretical powder reflections; and optionally associates each reflection with the Young’s modulus along its reciprocal-lattice normal when a compatible elastic tensor is available. Every output remains linked to the exact input structure, radiation settings, tensor source and coordinate frame, software versions, warnings, and file hashes. The result is a self-contained evidence bundle containing machine-readable provenance, tabular references, a spreadsheet, structured diagnostics, and a strict SHA-256 manifest. The package provides a command-line interface, Python API, desktop interface, and a deterministic offline verification example.

Statement of need

Candidate-phase screening in alloy and diffraction research commonly spans several independent tools. A researcher may translate an engineering alloy name into an element set, enumerate chemical subsystems, query a computed-materials database, download CIF records, inspect stability and symmetry metadata, calculate theoretical peaks, locate an elastic tensor, and then transfer selected values into a spreadsheet. Each individual operation is well supported, yet the hand-offs can separate a result from the structure setting, query conditions, radiation definition, tensor basis, warnings, or software version that produced it. This creates practical risks: duplicated candidates, ambiguous CIF-to-tensor pairing, coordinate-frame errors in directional properties, service failures recorded as missing data, and theoretical intensities compared with experimental peak areas under incompatible corrections.

DiffraScout addresses this workflow-level problem for materials researchers, beamline users, and students who need auditable theoretical references before experimental phase identification or refinement. The package preserves CIF identity (Hall, Allen, and Brown 1991), labels Materials Project structures and elastic tensors as computed records (Jain et al. 2013), and carries missing or unusable quantities forward as explicit states. Its scientific scope is deliberately bounded. Database candidates are hypotheses for evaluation, not experimentally confirmed phases. The diffraction output is a kinematic theoretical reference and does not perform Rietveld, Le Bail, or Pawley refinement, quantitative phase analysis, instrument calibration, background modelling, or preferred-orientation correction. This boundary allows the software to support experimental planning and interpretation without presenting a reference-generation workflow as an experimental inverse solution.

State of the field

The Materials Project provides computed structures and materials properties (Jain et al. 2013), and pymatgen supplies general materials-analysis objects and diffraction utilities (Ong et al. 2013). Gemmi provides CIF handling, crystallographic symmetry operations, and structure-factor calculations (Wojdyr 2022), while spglib provides an independent symmetry-search implementation (Togo, Shinohara, and Tanaka 2024). **Dans_Diffraction** offers broad crystal-property and diffraction simulation capabilities from CIF files (Porter and Prestipino 2023). Elasticity

provides general linear-elasticity and tensor analysis, including directional properties (Depriester and Kubler 2025). Experimental diffraction reduction and refinement are addressed by packages such as GSAS-II (Toby and Von Dreele 2013), and pyFAI specializes in detector geometry and azimuthal integration (Ashiotis et al. 2015).

DiffraScout uses these projects as data sources, numerical foundations, comparison tools, or downstream environments. Its contribution is the cross-stage contract that joins composition interpretation, subsystem queries, candidate deduplication, source acquisition, CIF identity, tensor pairing, theoretical reflection calculation, diagnostics, and export in one versioned research object. Direct diffraction simulators generally begin with a structure already selected by the user. Elasticity packages generally begin with a tensor whose coordinate basis is already known. Database clients return records without defining how a later diffraction table, directional modulus, and exported spreadsheet remain bound to the same cell setting. Refinement suites solve a different experimental problem.

General crystallographic, symmetry, database-client, and tensor algorithms remain in their established upstream projects. **DiffraScout** implements the workflow semantics that span those projects: fail-closed property pairing, explicit data states, stable output schemas, bounded numerical work, and non-destructive bundle creation. This separation provides a clear build-versus-contribute rationale while retaining upstream libraries for the domain algorithms they already implement well.

Software design

The package separates source providers, typed domain records, numerical calculation, orchestration, and export (Figure 1). A provider protocol returns normalized candidate records and source artifacts. The optional Materials Project adapter records query context and requests conventional-standard unit cells. Local CIF analysis remains available without a database client or network connection. This architecture keeps the offline scientific core deterministic and allows additional structure providers to be added without changing the diffraction engine.

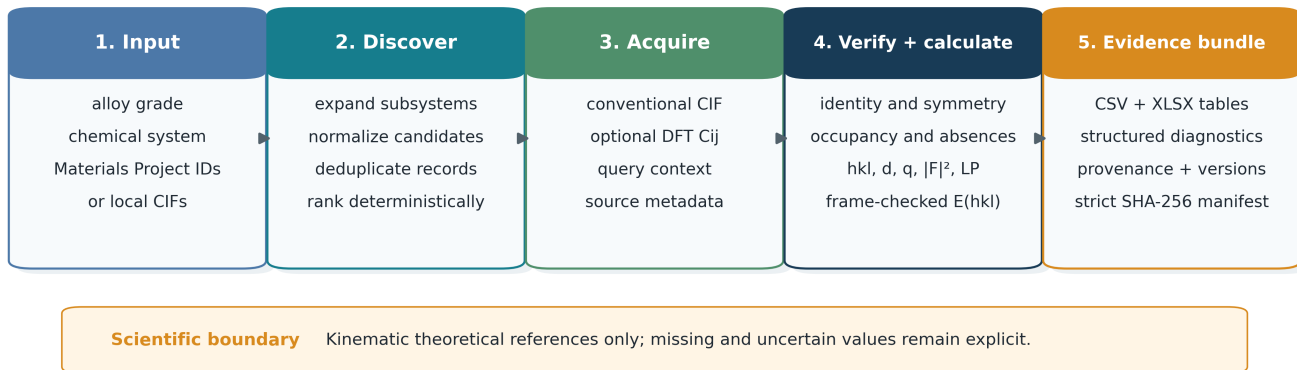


Figure 1: DiffraScout converts a composition request or local CIF collection into a verified evidence bundle. Each stage preserves identity, source, assumptions, warnings, and missing-data states.

CIF validation requires finite positive cell geometry and atomic sites. The software records whether a space-group assignment came from an explicit symbol, an International Tables number, Gemmi inference, or a warned fallback; spglib can provide an independent cross-check. Gemmi expands unit-cell sites, evaluates systematic absences, enumerates unique Miller-index families, and calculates X-ray structure factors. A dedicated structure-factor copy receives Gemmi’s crystallographic-occupancy conversion so that special-position multiplicity is not counted twice. The indexed table reports hkl , d , 2θ , $q = 2\pi/d$, multiplicity, $|F|^2$, and separate intensity channels with and without the explicitly defined Lorentz-polarization factor. A pseudo-Voigt profile is generated for visualization only. Profile-grid size and a conservative reciprocal-space search estimate are checked before large allocations.

Elasticity records contain a finite 6×6 stiffness matrix in GPa, source identity, nature-of-data label, and coordinate-frame declaration. Matrices are checked for symmetry, invertibility, conditioning, and positive definiteness. Under the engineering-shear Voigt convention, the directional modulus is

$$E(n) = [q(n)^T S q(n)]^{-1}, \quad S = C^{-1},$$

where n is the reciprocal-lattice normal of the reflection. Automatic sidecar pairing requires a unique CIF or material identifier. For Materials Project downloads, directional coupling uses the raw/POSCAR-format tensor paired with the conventional-standard CIF (Materials Project n.d.). IEEE-only records are retained with a `frame_transform_required` state and produce no directional modulus until a verified rotation is supplied.

The output directory is treated as a verifiable research bundle. A run writes into a sibling staging directory, generates all tables and metadata, creates a manifest, and verifies file hashes, byte sizes, safe paths, symbolic-link absence, and unlisted files before moving the bundle into place. Replacement of an existing bundle requires explicit authorization and a successful integrity check of the current bundle. A failed export or verification leaves the previous valid result untouched. Structured diagnostics preserve phase-specific failures while allowing independent phases to complete.

Research impact statement

The repository provides executable evidence for the package’s current scholarly significance. A packaged analytic suite performs 45 checks using simple-cubic, BCC, FCC, NaCl, and cubic-elasticity fixtures with closed-form selection rules, multiplicities, plane spacings, lattice structure factors, and directional moduli. The complete offline workflow uses a self-identifying synthetic $Fm\bar{3}m$ cell with $a = 4 \text{ \AA}$ and an isotropic cubic test tensor. For Cu $K\alpha$ radiation over $5\text{--}120^\circ$, it produces eight allowed reflection families, excludes the forbidden FCC (100) and (110) families, verifies d spacings, multiplicities, $q = 2\pi/d$, and $|F_{111}|^2 = (4f_{\text{Al}})^2$, and returns $E(hkl) = 110 \text{ GPa}$ for every direction. It writes a 5751-point profile and a 12-artifact bundle protected by independent SHA-256 entries (Figure 2). The fixtures are explicitly labelled as synthetic and carry no experimental material-property claim.

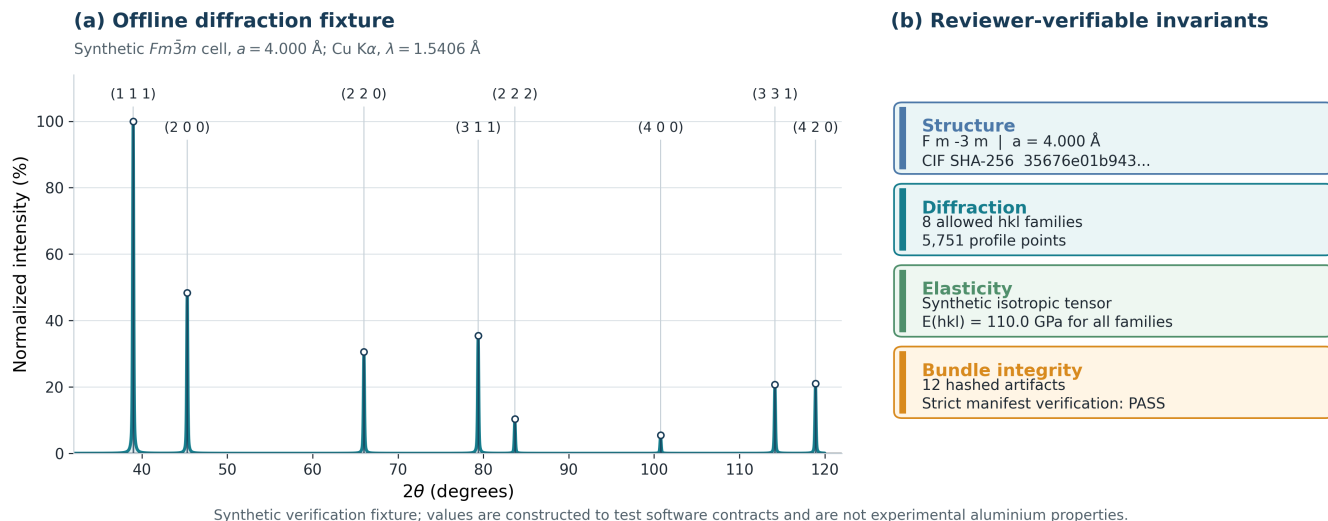


Figure 2: Offline verification output distributed with DiffractScout. (a) The indexed Cu $K\alpha$ powder profile for the synthetic FCC fixture, with all eight allowed reflection families labelled. (b) Selected invariants and bundle-integrity results checked by the automated suite.

The 68-test suite covers composition parsing, candidate selection, CIF and tensor validation, systematic absences, structure factors, intensity definitions, coordinate-frame boundaries, resource limits, transactional replacement, spreadsheet safety, and manifest tamper detection. A separate packaged benchmark performs 45 closed-form checks across simple-cubic, BCC, FCC, NaCl, and cubic directional-elasticity cases and writes a self-verifying evidence bundle. Continuous integration targets Python 3.10–3.13, Windows, macOS, a Linux graphical-interface smoke test, wheel installation, and Open Journals draft compilation. These materials let reviewers reproduce the declared numerical and provenance contracts without database credentials. Experimental accuracy and external adoption remain separate evidence. They are tracked through versioned validation cases and a machine-readable public evidence ledger, and the final submission will include only claims supported by those records.

AI usage disclosure

OpenAI GPT-5.6 Pro assisted integration planning, test scaffolding, documentation, figure production, and manuscript editing. The human author directed the scientific scope and retains responsibility for the architecture, numerical definitions, licensing, references, and manuscript claims. AI-assisted outputs were checked through source inspection, cited primary documentation, analytic fixtures, automated tests, bundle-integrity verification, and rendered-PDF review. Final submission requires the author’s explicit confirmation of complete human review.

Acknowledgements

The author acknowledges the maintainers and contributors of the Materials Project, pymatgen, Gemmi, spglib, GSAS-II, pyFAI, Dans_Diffraction, and Elasticipy projects, whose software and documentation support this workflow.

References

- Ashiotis, Giannis, Aurore Deschildre, Zubair Nawaz, Jonathan P. Wright, Dimitrios Karkoulis, Frédéric Emmanuel Picca, and Jérôme Kieffer. 2015. “The Fast Azimuthal Integration Python Library: pyFAI.” *Journal of Applied Crystallography* 48 (2): 510–19. <https://doi.org/10.1107/S1600576715004306>.
- Depriester, Dorian, and Régis Kubler. 2025. “Elasticipy: A Python Package for Linear Elasticity and Tensor Analysis.” *Journal of Open Source Software* 10 (115): 7940. <https://doi.org/10.21105/joss.07940>.
- Hall, S. R., F. H. Allen, and I. D. Brown. 1991. “The Crystallographic Information File (CIF): A New Standard Archive File for Crystallography.” *Acta Crystallographica Section A* 47 (6): 655–85. <https://doi.org/10.1107/S010876739101067X>.
- Jain, Anubhav, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, et al. 2013. “Commentary: The Materials Project: A Materials Genome Approach to Accelerating Materials Innovation.” *APL Materials* 1 (1): 011002. <https://doi.org/10.1063/1.4812323>.
- Materials Project. n.d. “Elastic Constants.” Accessed July 30, 2026. <https://docs.materialsproject.org/methodology/materials-methodology/elasticity>.
- Ong, Shyue Ping, William Davidson Richards, Anubhav Jain, Geoffroy Hautier, Michael Kocher, Shreyas Cholia, Dan Gunter, Vincent L. Chevrier, Kristin A. Persson, and Gerbrand Ceder. 2013. “Python Materials Genomics (pymatgen): A Robust, Open-Source Python Library for Materials Analysis.” *Computational Materials Science* 68: 314–19. <https://doi.org/10.1016/j.commatsci.2012.10.028>.
- Porter, Dan, and Carmelo Prestipino. 2023. “Dans_Diffraction: Version 3.0.0.” Zenodo. <https://doi.org/10.5281/zenodo.8106031>.
- Toby, Brian H., and Robert B. Von Dreele. 2013. “GSAS-II: The Genesis of a Modern Open-Source All Purpose Crystallography Software Package.” *Journal of Applied Crystallography* 46 (2): 544–49. <https://doi.org/10.1107/S0021889813003531>.
- Togo, Atsushi, Kohei Shinohara, and Isao Tanaka. 2024. “Spglib: A Software Library for Crystal Symmetry Search.” *Science and Technology of Advanced Materials: Methods* 4 (1): 2384822. <https://doi.org/10.1080/27660400.2024.2384822>.
- Wojdyr, Marcin. 2022. “GEMMI: A Library for Structural Biology.” *Journal of Open Source Software* 7 (73): 4200. <https://doi.org/10.21105/joss.04200>.